Question 1

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Industry Problem Solving Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.*
(BL4)

Dave's Taxonomy: Articulation (Level 4) Question Count: 5 **Total Marks: 40**
Duration :60 Minutes

Code of Conduct

I _Shaik Ashraf/192324088 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the student: _ __________

Assessment Tasks
Industry Problem Solving Task

1. **Retail Data Optimization Problem -** A retail chain operates both physical stores and an online platform. It generates large volumes of sales, inventory, and customer behavior data. The company wants real-time inventory tracking and demand forecasting. It also aims to deliver personalized marketing recommendations to customers. Design a big data architecture to support these requirements. Explain the tools, technologies, and analytics models you would use.

2. **Social Media Fraud Detection Problem -** A social media platform processes billions of user interactions daily. These include posts, comments, likes, and shares. The platform wants to detect fake accounts and misinformation quickly. Real-time or near real-time processing is required. Design a scalable big data solution for this use case. Explain the role of machine learning and stream processing frameworks.

3. **Government Data Platform Problem** A government agency is building a national data platform. It combines census, economic, and geospatial datasets. Different departments need access to shared and integrated data. The system must ensure interoperability across multiple agencies. Strict privacy and security requirements must be followed. Propose a governance and data management strategy.

4. **Banking Fraud Detection Problem** A bank processes millions of financial transactions every minute. It needs to detect fraudulent activities in real time. The system must analyze

historical and streaming transaction data. Low latency and high accuracy are critical requirements. Design a big data pipeline for fraud detection. Explain the technologies and algorithms you would implement.

5. **Agriculture Smart Farming Problem**
A smart farming system collects soil, weather, and crop data. Sensors and drones provide real-time

Question 1

field information. Farmers want insights to improve yield and resource usage. Data is generated from multiple distributed sources. Design a big data solution for precision agriculture. Explain how analytics can support decision-making.

Industry Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Industry Context	20%	Demonstrates strong understanding of real-world problem context	Good understanding	Basic understanding	Weak understanding
Application of Big Data Concepts	25%	Applies highly relevant big data ideas and tools	Mostly appropriate application	Partial application	Weak application
Solution Design	25%	Solution is practical, scalable, and well-reasoned	Reasonable solution	Basic solution with gaps	Unclear solution
Security / Risk Awareness	15%	Strong awareness of security and operational risks	Reasonable awareness	Limited awareness	Poor awareness
Clarity and Professionalism	15%	Well-structured and professional response	Generally clear	Moderately clear	Poorly presented

Total Marks Awarded:

Question 1 — Retail Data Optimization Problem

A retail chain operates physical stores and an online platform. It generates large volumes of sales, inventory and customer-behaviour data. The company needs real-time inventory tracking, demand forecasting and personalized marketing recommendations. Design a big data architecture and explain the tools, technologies and analytics models.

Answer

The retailer should build one integrated big data platform instead of keeping store, website and inventory information in separate systems. The design must support two speeds of work: streaming for stock and customer events that need an immediate response, and batch analytics for historical sales and forecasting. A cloud-based data lake provides the common storage layer, while a curated serving layer supplies dashboards and machine-learning applications.

Architecture and data flow

Point-of-sale systems, the website, mobile application and inventory software can publish events through Apache Kafka or Amazon Kinesis. Raw data can be stored in Amazon S3 or Azure Data Lake Storage. Apache Spark can clean, join and transform the data, while Spark Structured Streaming or Apache Flink can process stock changes and customer events with low latency. A common customer and product identifier is important because the same customer may appear differently in store and online records.

Analytics and business use

- Demand forecasting can use time-series models or XGBoost/Random Forest with sales history, promotions, holidays, price and location as features.
- Personalized recommendations can combine collaborative filtering with product attributes and recent clicks, searches, purchases and cart activity.
- Real-time inventory rules can trigger replenishment alerts when stock falls below a predicted requirement rather than relying only on fixed thresholds.
- Dashboards can show stock position, fast-moving products, forecast demand and campaign response so managers can act quickly.

Table 1 — Solution Design

Architecture component	Technology / method	Purpose
Sources	POS, website, app, inventory	Collect sales, stock and behaviour
Ingestion	Kafka / Kinesis	Capture real-time events
Storage	S3 / Azure Data Lake	Scalable raw and historical storage
Processing	Spark / Flink	Batch and stream processing
Models	Forecasting, XGBoost, recommender	Demand and customer prediction
Serving	Warehouse + BI dashboard	Business reporting and alerts

Result

This design creates a single view of products, inventory and customers. Streaming gives the retailer current stock visibility, while historical data improves demand forecasts. The same customer signals can support personalized marketing, making the platform useful for both daily operations and longer-term planning.

Question 1 — Flowchart

START

↓

Store + Online Data

↓

Kafka / Kinesis Ingestion

↓

Cloud Data Lake

↓

Spark / Flink Processing

↓

Forecasting + Recommendation Models

↓

Inventory / Marketing Decisions

↓

END

Question 2 — Social Media Fraud Detection Problem

A social media platform processes billions of posts, comments, likes and shares every day. It wants to identify fake accounts and misinformation quickly using real-time or near-real-time processing. Design a scalable solution and explain the role of machine learning and stream processing.

Answer

The platform needs a streaming-first design because a coordinated fake-account campaign can spread rapidly. A nightly batch process would identify the problem too late. The solution should evaluate events continuously, maintain historical information for model training and combine automated scoring with human review for high-impact decisions.

Streaming architecture

User actions can be published to partitioned Kafka topics. Apache Flink or Spark Structured Streaming can calculate features such as posting frequency, repeated content, sudden follower growth, unusual login behaviour and coordinated interactions. A cloud data lake can retain historical events for investigations and model training. Graph analytics is useful because fake accounts often operate as connected groups rather than isolated users.

Machine learning and risk scoring

- XGBoost, Random Forest or logistic regression can classify suspicious accounts using labelled historical cases.
- NLP models can examine text similarity and repeated misinformation patterns, while human reviewers handle uncertain or sensitive cases.
- Graph analysis can expose clusters of accounts that repeatedly share the same content or interact in an unnatural pattern.
- A risk engine can combine model scores with fixed rules and route high-risk events to review, restriction or further investigation.
- Model drift and false-positive rates must be monitored because attacker behaviour changes over time.

Table 2 — Solution Design

Component	Technology	Role
Event ingestion	Kafka	Partition high-volume interactions
Stream processing	Flink / Spark Streaming	Generate live risk features
Historical storage	Cloud data lake	Training and investigation
ML / NLP	XGBoost, RF, NLP	Score accounts and content
Graph analysis	Graph processing	Find coordinated networks
Decision layer	Rules + risk score	Trigger action or review

Result

The strongest solution combines streaming, machine learning and graph analysis. Streaming provides speed, historical storage provides context, and machine learning adapts detection to changing behaviour. Human oversight remains important because an automated system can make mistakes and legitimate users should not be penalized without appropriate review.

Question 2

Question 2 — Flowchart

START

↓

User Interaction

↓

Kafka Event Stream

↓

Flink / Spark Features

↓

ML + NLP + Graph Analysis

↓

Risk Score

↓

Review / Restrict / Allow

↓

END

Question 3 — Government Data Platform Problem

A government agency is building a national data platform combining census, economic and geospatial datasets. Different departments need shared integrated data, interoperability is required and strict privacy and security controls must be followed. Propose a governance and data-management strategy.

Answer

A national platform needs more than a large database. Different departments may use different definitions, identifiers, formats and access rules. The governance model should therefore create common standards without removing departmental ownership. Security and privacy must be built into the data lifecycle from collection to sharing and archival.

Governance and interoperability

The agency should maintain a central data catalogue containing dataset owners, definitions, refresh schedules, quality indicators, sensitivity levels and permitted uses. Common identifiers, schemas and data dictionaries should be established for shared entities such as locations, administrative regions and economic measures. APIs and open data formats can make datasets interoperable without encouraging uncontrolled file copies. Data lineage should show the source and transformations applied to every important dataset.

Security, privacy and quality controls

- Use role-based or attribute-based access, strong authentication, encryption and detailed audit logs.
- Apply data minimization, masking, pseudonymization or anonymization where personal information does not need to remain identifiable.
- Run automated checks for completeness, consistency, validity, duplication and timeliness before data enters the shared analytical layer.
- Assign data owners and data stewards so responsibility for quality, access and definitions is clear.
- Review permissions regularly and retain audit evidence for sensitive data access and sharing.

Table 3 — Solution Design

Governance area	Strategy	Benefit
Ownership	Data owners + stewards	Clear accountability
Metadata	Central catalogue + dictionary	Common understanding
Interoperability	Schemas, APIs, open formats	Cross-agency integration
Quality	Automated validation	Reliable datasets
Security	RBAC/ABAC, MFA, encryption	Controlled access
Privacy	Minimization and masking	Lower exposure risk
Lineage	Source-to-use tracking	Transparency and trust

Result

A federated governance model is appropriate: departments retain operational ownership, while the national platform provides shared standards and controlled access. Interoperability should make information easier to use across agencies, not make sensitive information freely available. Governance, privacy, security and lineage therefore need to operate together.

Question 3

Question 3 — Flowchart

START

↓

Department Data Sources

↓

Common Standards + Catalogue

↓

Quality Validation

↓

Secure Integration Layer

↓

Controlled Cross-Agency Access

↓

Audit + Governance Review

↓

END

Question 4

Question 4 — Banking Fraud Detection Problem

A bank processes millions of financial transactions every minute and needs real-time fraud detection. The system must combine historical and streaming data, with low latency and high accuracy. Design a big data pipeline and explain the technologies and algorithms.

Answer

Banking fraud detection is a low-latency problem, but it is also a historical learning problem. Each transaction must be scored quickly, while past transactions provide the behavioural patterns needed to identify unusual activity. A layered pipeline should therefore separate event ingestion, feature generation, model scoring, decision logic and historical analysis.

Real-time fraud pipeline

Transactions can enter through secure APIs and be published to Kafka. Apache Flink or Spark Structured Streaming can calculate features such as transaction amount compared with normal spending, transaction frequency, merchant risk, device history and unusual location changes. A low-latency feature store can supply current customer information to the scoring service. The final decision should combine the model probability with business rules so known high-risk cases can be handled immediately.

Models and operational controls

- XGBoost or Random Forest can classify transactions using labelled historical fraud cases and behavioural features.
- Anomaly detection can identify new fraud patterns that are not yet well represented in labelled training data.
- Graph or sequence analysis can connect accounts, devices, merchants and transactions to reveal coordinated fraud.
- Historical data should remain in a data lake or warehouse for investigation, retraining and trend analysis.
- Precision, recall, false-positive rate, model drift and scoring latency should be monitored continuously.

Table 4 — Solution Design

Stage	Technology / method	Purpose
Transaction intake	Secure API + Kafka	Receive and distribute events
Streaming	Flink / Spark	Generate live features
Feature store	Low-latency store	Serve current customer features
Scoring	XGBoost / RF / anomaly model	Estimate fraud risk
Decision	Rules + model score	Approve, challenge or decline
Historical layer	Data lake + Spark	Training and investigation
Monitoring	Model + system monitoring	Track accuracy and latency

Result

A hybrid pipeline gives the bank both speed and analytical depth. Streaming detects suspicious transactions before losses grow, machine learning adapts to complex patterns, and deterministic rules provide immediate protection for known threats. A feedback loop is necessary because confirmed fraud becomes new training data and fraud patterns change over time.

Question 4 — Flowchart

START

↓

Transaction Event

↓

Kafka Ingestion

↓

Flink / Spark Feature Generation

↓

Fraud Model + Rules

↓

Risk Decision

↓

Approve / Challenge / Decline

↓

Feedback + Retraining

↓

END

Question 5 — Agriculture Smart Farming Problem

A smart farming system collects soil, weather and crop data. Sensors and drones provide real-time field information, while farmers want better yield and resource-use decisions. Data comes from multiple distributed sources. Design a big data solution for precision agriculture and explain how analytics supports decision-making.

Answer

Precision agriculture works best when information from sensors, weather services, drones and crop records is brought together at field level. These sources have different data sizes and update rates, so the platform should combine edge or IoT collection, streaming analytics and historical storage. The goal is not simply to collect measurements but to turn them into practical recommendations.

Smart-farming architecture

Soil sensors can report moisture, temperature and pH through an IoT gateway. MQTT or a cloud IoT service can handle lightweight device communication, while Kafka can provide scalable event ingestion. A data lake can store sensor history, weather data, drone imagery and crop records. Stream processing can detect immediate irrigation or weather conditions, while batch processing can study seasonal trends and train predictive models.

Analytics and decision support

- Yield models can combine soil conditions, weather, crop type and historical production to estimate expected output.
- Time-series analysis can identify moisture trends and recommend irrigation when field conditions require it.
- Computer vision can analyse drone images for crop stress, disease symptoms, weeds or uneven plant growth.
- Weather and crop data can be combined to estimate disease risk and support preventive action.
- Resource-optimization models can guide where water and fertilizer should be applied, reducing unnecessary use.
- Mobile dashboards and alerts should present field-level recommendations instead of forcing farmers to interpret raw sensor readings.

Table 5 — Solution Design

Data source	Information	Analytics use
Soil sensors	Moisture, pH, temperature	Irrigation and soil analysis
Weather	Rainfall, humidity, temperature	Yield and disease prediction
Drone imagery	Crop and field images	Stress, disease and weed detection
Crop records	Crop type, planting date, yield	Forecasting and comparison
IoT gateway	Live measurements	Fast collection and filtering
Data platform	Integrated history + streams	Cross-source analytics
Dashboard	Alerts and recommendations	Farmer decisions

Result

The main value of smart farming is the conversion of distributed observations into timely field-level decisions. By combining soil, weather, crop and drone information, the system can improve irrigation, fertilizer use, disease management and yield forecasting. Recommendations should remain understandable so that farmers can act on them with confidence.

Question 5 — Flowchart

START

↓

Sensors + Weather + Drone Data

↓

IoT Gateway / Kafka

↓

Data Lake

↓

Stream + Batch Analytics

↓

Yield / Irrigation / Disease Models

↓

Farmer Recommendation

↓

END